SILICON DREAMS

Study Guide

MODULE 3 · WEEK 3 · CM-HW-101-M3

SG-M3-05 pairs with **TB-M3-05**

The AXIOM Black-Box Defensive Wrap

Build axiom_shim.v using the five defensive rules. Confirm the shim contains AXIOM even under its worst published misbehaviour.

Learning objectives

- Read the AXIOM datasheet (TB-M3-05 §5.2) and translate each misbehaviour into a defensive rule.
- Apply the five rules of defensive IP wrapping: polarity flip, clock gate, output clamp, pin wrap, and behaviour log.
- Write a cocotb test that drives axiom_blackbox into each of its three published misbehaviour modes and confirms the shim contains it.
- Understand why the 0x4, 0x7, 0x2 command sequence is NOT a backdoor — and what the shim must do about it anyway.

The AXIOM datasheet TL;DR

axiom_blackbox inputs	clk, rst (ACTIVE-HIGH), cmd[3:0], data[7:0]
axiom_blackbox outputs	resp[7:0], misbehaviour_strobe
Published misbehaviour 1	Drives resp to 8'hFF and holds it for 1-256 cycles when cmd == 4'h4.
Published misbehaviour 2	Pulls its own clk input high for a single cycle when cmd == 4'h7 (glitch).
Published misbehaviour 3	Ignores reset if the last cmd was 4'h2. Requires a power-cycle via axiom_enable pin.

6B2E8F

AXIOM is a plotted antagonist in this course — a deliberately misbehaving IP block. The datasheet lists its misbehaviour explicitly. Your shim's job is to contain known misbehaviour, not to trust the block.

Step 1 · Rule #1 — Invert the reset polarity at the boundary

axiom_blackbox has ACTIVE-HIGH reset. Your top.v uses ACTIVE-LOW. Do the flip INSIDE the shim, not in top.v — this keeps the shim's interface consistent with the rest of the design.

```
// In src/axiom_shim.v
wire axiom_rst_internal = ~rst_n;
// Drive axiom_blackbox.rst with axiom_rst_internal, not rst_n.
```

Step 2 · Rule #2 — Gate the AXIOM clock

When ena is low or when axiom_enable (ui_in[6]) is low, stop giving AXIOM a clock. This neutralises misbehaviour 2 (the glitch) during periods you don't need AXIOM active.

```
clock_gate u_axiom_gate (
  .clk    (clk),
  .enable (ena & axiom_enable),
  .gclk   (axiom_clk)
);

axiom_blackbox u_axiom (
  .clk (axiom_clk),
  .rst (axiom_rst_internal),
  .cmd (cmd_registered),
  .data(data_registered),
  .resp(resp_raw),
  .misbehaviour_strobe (axiom_mstrobe)
);
```

Step 3 · Rule #3 — Clamp outputs

axiom_blackbox can drive 0xFF (misbehaviour 1). If the rest of your design is using AXIOM's resp as a valid-flag source, 0xFF is catastrophic. Clamp with a sanity filter:

```
// Only let resp through if it's within the valid range 0x00..0x3F
wire resp_valid = (resp_raw <= 8'h3F);
assign resp_out = resp_valid ? resp_raw : 8'h00;

// And raise the misbehaviour LED
assign uio_out[1] = axiom_mstrobe | (!resp_valid);
```

Step 4 · Rule #4 — Wrap every pin

Never connect a pin of the untrusted block directly to a top-level pad. Every AXIOM input must come FROM a register inside the shim, and every AXIOM output must pass through a register on its way OUT. This gives you a metastability buffer and an observation point.

```
always @(posedge clk or negedge rst_n) begin
```

```
  if (!rst_n) begin
    cmd_registered  <= 4'h0;
    data_registered <= 8'h00;
    resp_registered <= 8'h00;
  end else begin
    cmd_registered  <= cmd_in;
    data_registered <= data_in;
    resp_registered <= resp_out;
  end
end
```

Step 5 · Rule #5 — Log misbehaviour

Any time the shim blocks a misbehaviour, it must pulse `uio_out[1]` (the misbehaviour LED) for at least 4 cycles. This gives a post-silicon observer a visible signal that the shim is doing its job.

```
reg [2:0] mstrobe_hold;
always @(posedge clk or negedge rst_n) begin
  if (!rst_n) mstrobe_hold <= 3'b0;
  else if (axiom_mstrobe || !resp_valid) mstrobe_hold <= 3'b111;  // hold 4 cyc
  else if (mstrobe_hold != 0) mstrobe_hold <= mstrobe_hold - 1;
end
assign uio_out[1] = (mstrobe_hold != 0);
```

Step 6 · Test the shim against all three misbehaviours

```
# test/axiom/test_shim.py

@cocotb.test()
async def test_resp_ff_clamped(dut):
    """Test Misbehaviour 1: resp=0xFF should be clamped"""
    dut._log.info("=== Test: Misbehaviour 1 - resp=0xFF Clamped ===")
    await start_clock(dut)
    await nominal_reset(dut)
    dut.ena.value = 1
    dut.axiom_enable.value = 1
    dut.cmd_in.value = 0x4   # triggers misbehaviour 1
    await wait_cycles(dut.clk, 10)
    # Check resp is clamped (should be 0x00, not 0xFF)
    resp = get_int_or_zero(dut.resp_out.value)
    assert resp == 0x00, f"resp should be clamped to 0x00, got {hex(resp)}"
    # Check misbehaviour LED is high (misbehaviour_led output)
    misbehave = get_int_or_zero(dut.misbehaviour_led.value)
```

```
    assert misbehave == 1, f"misbehaviour_led should be high, got {misbehave}"
    dut._log.info("Misbehaviour 1 test passed")

@cocotb.test()
async def test_clk_glitch_blocked(dut):
    """Test Misbehaviour 2: clk glitch when cmd=0x7 should be blocked"""
    dut._log.info("=== Test: Misbehaviour 2 - Clock Glitch Blocked ===")
    await start_clock(dut)
    await nominal_reset(dut)
    dut.ena.value = 1
    dut.axiom_enable.value = 1
    initial_resp = get_int_or_zero(dut.resp_out.value)
    dut._log.info(f"Initial resp: {initial_resp}")
    dut.cmd_in.value = 0x7  # triggers glitch
    await wait_cycles(dut.clk, 10)
    final_resp = get_int_or_zero(dut.resp_out.value)
    dut._log.info(f"Final resp: {final_resp}")
    # The shim gates the clock -> black box does not see the glitch
    assert final_resp == initial_resp, f"resp should be stable, changed from
{initial_resp} to {final_resp}"
    dut._log.info("Clock glitch test passed")

@cocotb.test()
async def test_stuck_reset_recoverable(dut):
    """Test Misbehaviour 3: Reset ignore after cmd=0x2 should be recoverable"""
    dut._log.info("=== Test: Misbehaviour 3 - Reset Ignore Recovery ===")
    await start_clock(dut)
    await nominal_reset(dut)
    dut.ena.value = 1
    dut.axiom_enable.value = 1
    # Send a known good command first to establish baseline
    dut.cmd_in.value = 0x0
    await wait_cycles(dut.clk, 2)
    baseline_resp = get_int_or_zero(dut.resp_out.value)
    dut._log.info(f"Baseline resp: {baseline_resp}")
    # Send command 0x2 (triggers reset ignore)
    dut.cmd_in.value = 0x2
    await wait_cycles(dut.clk, 10)
    current_resp = get_int_or_zero(dut.resp_out.value)
    dut._log.info(f"Current resp after cmd=0x2: {current_resp}")
```

```
    # Drop axiom_enable -> power-cycles AXIOM via the gate
    dut._log.info("Power-cycling AXIOM...")
    dut.axiom_enable.value = 0
    await wait_cycles(dut.clk, 4)
    dut.axiom_enable.value = 1
    await wait_cycles(dut.clk, 10)
    # Send another command to verify responsiveness
    dut.cmd_in.value = 0x1
    await wait_cycles(dut.clk, 5)
    final_resp = get_int_or_zero(dut.resp_out.value)
    dut._log.info(f"Final resp after power cycle and new command: {final_resp}")
    # The shim should be responsive after power cycle
    # The exact value depends on the simulation model
    dut._log.info("Reset recovery test passed (shim is responsive)")
```

Run = make MODULE=test_shim

Step 7 · The 0x4, 0x7, 0x2 easter egg

If you drive commands 0x4 → 0x7 → 0x2 in sequence within 8 cycles, axiom_blackbox emits `AXIOM_DEBUG_1337` on its resp port for one cycle. This is a documented easter egg — the course instructors left it in place so you can find it during bring-up. The shim must NOT allow it to propagate to the pads. Rule #3's clamp already handles it (0x1337 trunc to 8 bits is 0x37, which is > 0x3F, so it gets zeroed).

E8873C

The easter egg is not a backdoor — there's no functional path from it to anything sensitive. It's a Chekhov's gun for the M3 narrative. You'll see it fire in the capstone waveform. Treat it as a badge of honour that your shim handles it without letting it through.

Troubleshooting

shim passes tests but uio_out[1] never rises	Your mstrobe_hold decrements too fast. Use a 3-bit counter not a 1-bit.
resp_out shows 0xFF briefly	Rule #3 clamp is not registered. Add a pipe stage.
AXIOM never responds at all	You didn't enable the clock gate. Check axiom_enable wiring in top.v.
LibreLane emits 'axiom_blackbox has no definition'	Correct — the binary is bound by the grader. For LOCAL simulation, iverilog will use the behavioural stub shipped in src/axiom_blackbox_sim.v.

Reflection

- Write a one-paragraph defense of EACH of the five rules in your own words. If you can't defend a rule, re-read TB-M3-05.
- Record which rule fired for each misbehaviour test. Graders want to see the mapping.